

Enterprise Web Application Assignment 3

List of Experiments (Exception Handling & Debugging) - UCS553 (Week 5)

Q1. Exception Hierarchy Demonstration

Create a program that demonstrates the hierarchy of common exceptions such as `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`, and `NumberFormatException`.

1. Question1.java

```
package q1;

public class Question1 {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        }

        try {
            String str = null;
            System.out.println(str.length());
        } catch (NullPointerException e) {
            System.out.println("Caught NullPointerException: " + e.getMessage());
        }

        try {
            int[] arr = new int[3];
            System.out.println(arr[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Caught ArrayIndexOutOfBoundsException: " + e.getMessage());
        }

        try {
            int num = Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            System.out.println("Caught NumberFormatException: " + e.getMessage());
        }
    }
}
```

Q1 Output

```
Terminal - Q1 Output

shreyaas@shreyaas-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp
bin q1.Question1
Caught ArithmeticException: / by zero
Caught NullPointerException: Cannot invoke "String.length()" because "<local1>" is null
Caught ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
Caught NumberFormatException: For input string: "abc"
```

Q2. Nested Try-Catch Blocks

Write a program containing nested try-catch blocks. Generate different exceptions in the inner and outer blocks.

1. Question2.java

```
package q2;

public class Question2 {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block started.");
            try {
                System.out.println("Inner try block started.");
                int res = 50 / 0;
            } catch (ArithmeticException e) {
                System.out.println("Inner catch: ArithmeticException caught -> " + e.getMessage());
            }
            int[] arr = new int[2];
            arr[10] = 5;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Outer catch: ArrayIndexOutOfBoundsException caught -> " + e.getMessage());
        }
        catch (Exception e) {
            System.out.println("Outer catch: General Exception caught.");
        }
        System.out.println("Execution finished.");
    }
}
```

Q2 Output



```
Terminal - Q2 Output

shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp
bin q2.Question2
Outer try block started.
Inner try block started.
Inner catch: ArithmeticException caught -> / by zero
Outer catch: ArrayIndexOutOfBoundsException caught -> Index 10 out of bounds for length 2
Execution finished.
```

Q3. Calculator with Dedicated Exception Handlers

Develop a calculator that accepts two numbers and an operator (+, -, *, /). It should be able to handle the following exceptions: Division by zero, Invalid numeric input, Invalid operators. Use separate exception handlers for different errors.

1. Question3.java

```
package q3;

import java.util.Scanner;

public class Question3 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Enter first number: ");
            double num1 = Double.parseDouble(scanner.nextLine());
            System.out.print("Enter operator (+, -, *, /): ");
            String op = scanner.nextLine();
            System.out.print("Enter second number: ");
            double num2 = Double.parseDouble(scanner.nextLine());

            if (!op.equals("+") && !op.equals("-") && !op.equals("*") && !op.equals("/")) {
                throw new IllegalArgumentException("Invalid operator entered.");
            }
            if (op.equals("/") && num2 == 0) {
                throw new ArithmeticException("Division by zero is not allowed.");
            }

            double result = 0;
            switch (op) {
                case "+": result = num1 + num2; break;
                case "-": result = num1 - num2; break;
                case "*": result = num1 * num2; break;
                case "/": result = num1 / num2; break;
            }
            System.out.println("Result: " + result);
        } catch (NumberFormatException e) {
            System.out.println("Error: Invalid numeric input.");
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Q3 Output

```
Terminal - Q3 Output

shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp
bin q3.Question3
Enter first number: 10
Enter operator (+, -, *, /): /
Enter second number: 0
Error: Division by zero is not allowed.

shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp
bin q3.Question3
Enter first number: abc
Error: Invalid numeric input.
```

Q4. Student Result-Processing Program

Create a student result-processing program that accepts marks for several subjects. Marks must be between 0 and 100. Create a custom InvalidMarksException. Throw the exception when invalid marks are entered. Calculate total, percentage, and grade only when all input is valid. Display a meaningful error message.

1. Question4.java

```
package q4;

class InvalidMarksException extends Exception {
    public InvalidMarksException(String message) {
        super(message);
    }
}

public class Question4 {
    public static void validateAndProcess(double[] marks) throws InvalidMarksException {
        double total = 0;
        for (double mark : marks) {
            if (mark < 0 || mark > 100) {
                throw new InvalidMarksException("Invalid mark: " + mark + ". Marks must be between 0 and 100.");
            }
            total += mark;
        }
        double percentage = total / marks.length;
        String grade;
        if (percentage >= 90) grade = "A";
        else if (percentage >= 75) grade = "B";
        else if (percentage >= 50) grade = "C";
        else grade = "F";

        System.out.println("Total Marks: " + total);
        System.out.println("Percentage: " + percentage + "%");
        System.out.println("Grade: " + grade);
    }

    public static void main(String[] args) {
        double[] validMarks = { 85.5, 90.0, 78.0 };
        double[] invalidMarks = { 85.5, 105.0, 78.0 };

        System.out.println("Processing valid marks:");
        try {
            validateAndProcess(validMarks);
        } catch (InvalidMarksException e) {
            System.out.println("Error: " + e.getMessage());
        }

        System.out.println("\nProcessing invalid marks:");
        try {
            validateAndProcess(invalidMarks);
        } catch (InvalidMarksException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Q4 Output Terminal - Q4 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp  
bin q4.Question4
```

```
Processing valid marks:
```

```
Total Marks: 253.5
```

```
Percentage: 84.5%
```

```
Grade: B
```

```
Processing invalid marks:
```

```
Error: Invalid mark: 105.0. Marks must be between 0 and 100.
```

Q5. Banking Application with Exception Propagation

Implement a simple banking application supporting deposit and withdrawal. Create a custom `InsufficientBalanceException`. Create a custom `InvalidAmountException`. Prevent withdrawal when the balance is insufficient. Prevent zero or negative deposits/withdrawals. Demonstrate exception propagation from one method to another. Display the account balance after successful transactions. Add `AccountNotFoundException` for invalid account numbers.

1. Question5.java

```
package q5;

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) { super(msg); }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String msg) { super(msg); }
}

class AccountNotFoundException extends Exception {
    public AccountNotFoundException(String msg) { super(msg); }
}

class BankAccount {
    private String accountNumber;
    private double balance;

    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }

    public String getAccountNumber() { return accountNumber; }
    public double getBalance() { return balance; }

    public void deposit(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Deposit amount must be greater than zero.");
        }
        balance += amount;
    }

    public void withdraw(double amount) throws InvalidAmountException, InsufficientBalanceException {
        if (amount <= 0) {
            throw new InvalidAmountException("Withdrawal amount must be greater than zero.");
        }
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance. Available: " + balance);
        }
        balance -= amount;
    }
}

class BankService {
    private BankAccount account;

    public BankService(BankAccount account) {
        this.account = account;
    }

    public void processDeposit(String accNo, double amount) throws AccountNotFoundException, InvalidAmountException {
        if (!account.getAccountNumber().equals(accNo)) {
            throw new AccountNotFoundException("Account number " + accNo + " not found.");
        }
        account.deposit(amount);
    }
}
```

```
    public void processWithdrawal(String accNo, double amount) throws AccountNotFoundException, Invalid
AmountException, InsufficientBalanceException {
        if (!account.getAccountNumber().equals(accNo)) {
            throw new AccountNotFoundException("Account number " + accNo + " not found.");
        }
        account.withdraw(amount);
    }
}

public class Question5 {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("ACC123", 500.0);
        BankService service = new BankService(acc);

        try {
            service.processDeposit("ACC123", 200.0);
            System.out.println("Deposit successful. Balance: " + acc.getBalance());

            service.processWithdrawal("ACC123", 100.0);
            System.out.println("Withdrawal successful. Balance: " + acc.getBalance());

            service.processWithdrawal("ACC999", 50.0);
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            service.processWithdrawal("ACC123", 1000.0);
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Q5 Output

```
Terminal - Q5 Output

shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp
bin q5.Question5
Deposit successful. Balance: 700.0
Withdrawal successful. Balance: 600.0
Error: Account number ACC999 not found.
Error: Insufficient balance. Available: 600.0
```

Q6. Login System with Exception-Based Validation & Account Lockout

Develop a login system with exception-based validation. Create custom exceptions such as: InvalidUsernameException, InvalidPasswordException, AccountLockedException, Allow only a fixed number of failed login attempts. Lock the account after the maximum attempts. Use try-catch-finally appropriately. Display different messages for different failures.

1. Question6.java

```
package q6;

class InvalidUsernameException extends Exception {
    public InvalidUsernameException(String msg) { super(msg); }
}

class InvalidPasswordException extends Exception {
    public InvalidPasswordException(String msg) { super(msg); }
}

class AccountLockedException extends Exception {
    public AccountLockedException(String msg) { super(msg); }
}

class LoginSystem {
    private final String validUsername = "admin";
    private final String validPassword = "password123";
    private int failedAttempts = 0;
    private final int MAX_ATTEMPTS = 3;
    private boolean isLocked = false;

    public void login(String username, String password) throws InvalidUsernameException, InvalidPasswordException, AccountLockedException {
        if (isLocked) {
            throw new AccountLockedException("Account is locked due to too many failed attempts.");
        }
        if (!validUsername.equals(username)) {
            registerFailedAttempt();
            throw new InvalidUsernameException("Invalid username.");
        }
        if (!validPassword.equals(password)) {
            registerFailedAttempt();
            throw new InvalidPasswordException("Invalid password.");
        }
        failedAttempts = 0;
        System.out.println("Login successful! Welcome, " + username + ".");
    }

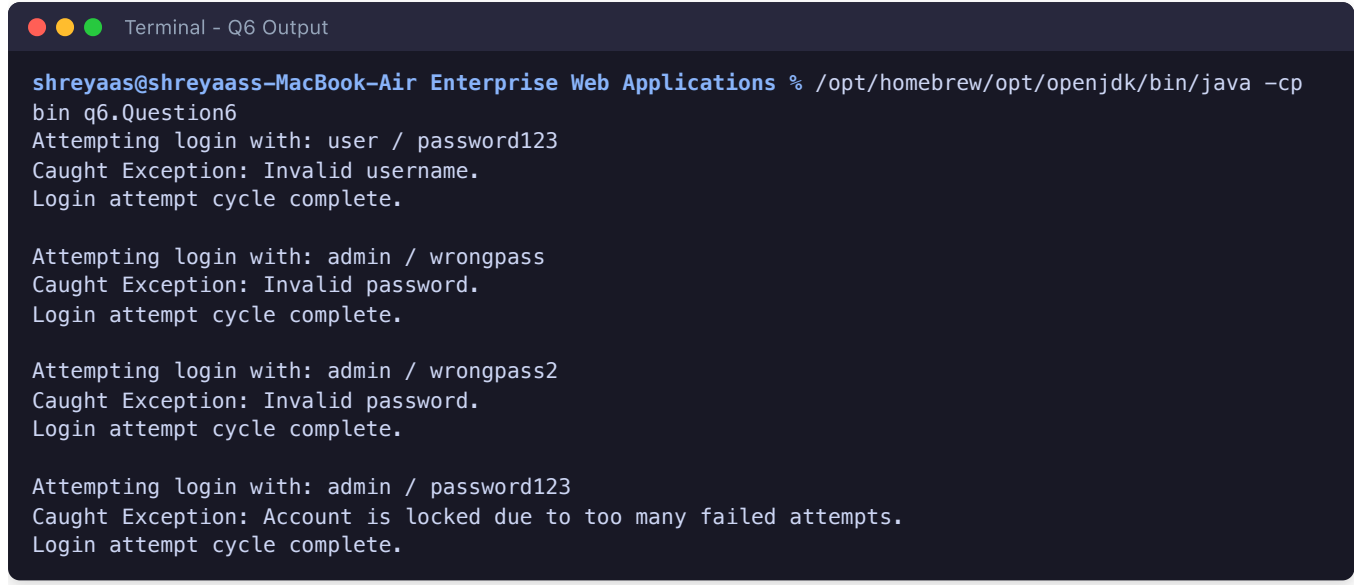
    private void registerFailedAttempt() {
        failedAttempts++;
        if (failedAttempts >= MAX_ATTEMPTS) {
            isLocked = true;
        }
    }
}

public class Question6 {
    public static void main(String[] args) {
        LoginSystem system = new LoginSystem();
        String[][] credentials = {
            { "user", "password123" },
            { "admin", "wrongpass" },
            { "admin", "wrongpass2" },
            { "admin", "password123" }
        };

        for (String[] cred : credentials) {
            try {
                System.out.println("Attempting login with: " + cred[0] + " / " + cred[1]);
            }
        }
    }
}
```

```
        system.login(cred[0], cred[1]);
    } catch (InvalidUsernameException | InvalidPasswordException | AccountLockedException e) {
        System.out.println("Caught Exception: " + e.getMessage());
    } finally {
        System.out.println("Login attempt cycle complete.\n");
    }
}
}
```

Q6 Output



```
Terminal - Q6 Output

shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp
bin q6.Question6
Attempting login with: user / password123
Caught Exception: Invalid username.
Login attempt cycle complete.

Attempting login with: admin / wrongpass
Caught Exception: Invalid password.
Login attempt cycle complete.

Attempting login with: admin / wrongpass2
Caught Exception: Invalid password.
Login attempt cycle complete.

Attempting login with: admin / password123
Caught Exception: Account is locked due to too many failed attempts.
Login attempt cycle complete.
```

Q7. Shopping Cart Application with Complete Exception Hierarchy

Design a small shopping/cart application that uses a complete exception hierarchy: ApplicationException -> ProductException (ProductNotFoundException, OutOfStockException), PaymentException (InvalidPaymentException, InsufficientFundsException), OrderException (EmptyCartException). Implement the appropriate operations like add/remove, payment, search products etc. Use appropriate exception handlers and debug the program using breakpoints and variable inspection.

1. Question7.java

```
package q7;

import java.util.ArrayList;
import java.util.List;

class ApplicationException extends Exception {
    public ApplicationException(String msg) { super(msg); }
}

class ProductException extends ApplicationException {
    public ProductException(String msg) { super(msg); }
}

class ProductNotFoundException extends ProductException {
    public ProductNotFoundException(String msg) { super(msg); }
}

class OutOfStockException extends ProductException {
    public OutOfStockException(String msg) { super(msg); }
}

class PaymentException extends ApplicationException {
    public PaymentException(String msg) { super(msg); }
}

class InvalidPaymentException extends PaymentException {
    public InvalidPaymentException(String msg) { super(msg); }
}

class InsufficientFundsException extends PaymentException {
    public InsufficientFundsException(String msg) { super(msg); }
}

class OrderException extends ApplicationException {
    public OrderException(String msg) { super(msg); }
}

class EmptyCartException extends OrderException {
    public EmptyCartException(String msg) { super(msg); }
}

class Product {
    private String id;
    private String name;
    private double price;
    private int stock;

    public Product(String id, String name, double price, int stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }
    public int getStock() { return stock; }
```

```
    public void reduceStock(int qty) { this.stock -= qty; }
}

class ShoppingCart {
    private List<Product> items = new ArrayList<>();

    public void addProduct(Product product) throws OutOfStockException {
        if (product.getStock() <= 0) {
            throw new OutOfStockException("Product " + product.getName() + " is out of stock.");
        }
        items.add(product);
        product.reduceStock(1);
    }

    public void removeProduct(Product product) {
        items.remove(product);
    }

    public double calculateTotal() throws EmptyCartException {
        if (items.isEmpty()) {
            throw new EmptyCartException("Cannot checkout: Cart is empty.");
        }
        double total = 0;
        for (Product p : items) {
            total += p.getPrice();
        }
        return total;
    }

    public void clearCart() {
        items.clear();
    }
}

public class Question7 {
    public static Product searchProduct(List<Product> catalog, String id) throws ProductNotFoundException {
        for (Product p : catalog) {
            if (p.getId().equalsIgnoreCase(id)) {
                return p;
            }
        }
        throw new ProductNotFoundException("Product with ID " + id + " not found.");
    }

    public static void processPayment(double total, double balance, String paymentMethod) throws PaymentException {
        if (!paymentMethod.equalsIgnoreCase("CREDIT") && !paymentMethod.equalsIgnoreCase("DEBIT")) {
            throw new InvalidPaymentException("Payment method " + paymentMethod + " is invalid.");
        }
        if (balance < total) {
            throw new InsufficientFundsException("Insufficient funds. Required: " + total + ", Available: " + balance);
        }
        System.out.println("Payment of $" + total + " successful via " + paymentMethod + ".");
    }

    public static void main(String[] args) {
        List<Product> catalog = new ArrayList<>();
        catalog.add(new Product("P101", "Laptop", 1000.0, 2));
        catalog.add(new Product("P102", "Mouse", 25.0, 0));

        ShoppingCart cart = new ShoppingCart();

        try {
            Product p1 = searchProduct(catalog, "P101");
            cart.addProduct(p1);
            System.out.println("Added " + p1.getName() + " to cart.");

            Product p2 = searchProduct(catalog, "P102");
            cart.addProduct(p2);
        }
    }
}
```

```
    } catch (ProductException e) {  
        System.out.println("Product Error: " + e.getMessage());  
    }  
  
    try {  
        double total = cart.calculateTotal();  
        processPayment(total, 500.0, "CREDIT");  
    } catch (ApplicationException e) {  
        System.out.println("Checkout Error: " + e.getMessage());  
    }  
}
```

Q7 Output

Terminal - Q7 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp  
bin q7.Question7  
Added Laptop to cart.  
Product Error: Product Mouse is out of stock.  
Checkout Error: Insufficient funds. Required: 1000.0, Available: 500.0
```